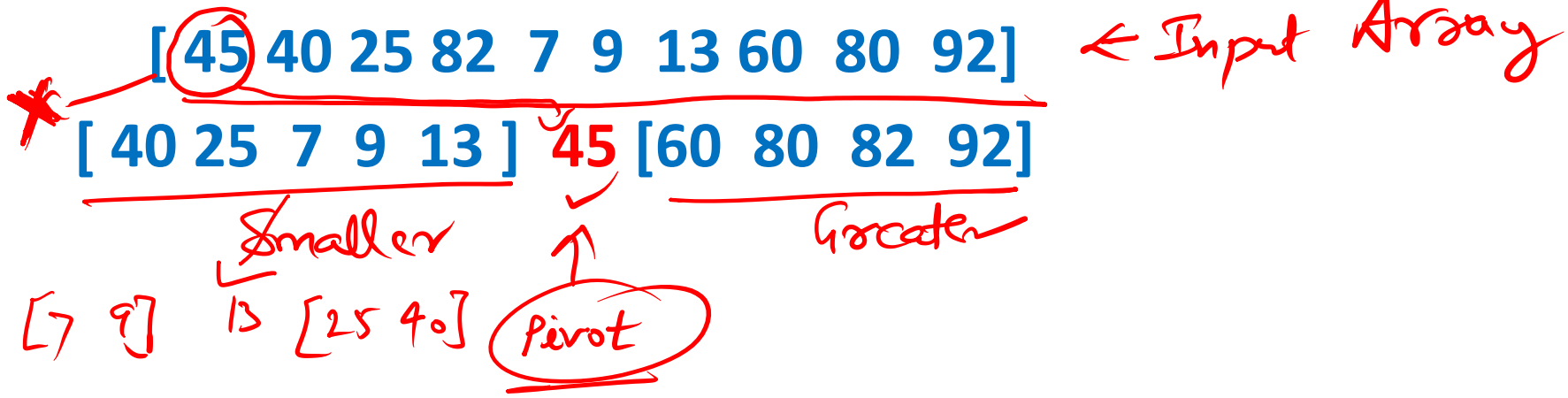
Quicksort

Similar to Merge Sort, But not always divide into equal halves.

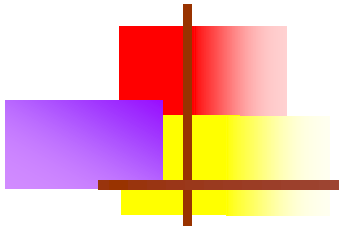
Basic Idea in sorting an array

- Choose an element x in the array as pivot
- Place x in the array A such that
  - All elements to the left of x are ≤ x
  - All elements to the right of x are > x
  - So x is now in its proper position in the final sorted array
- Recursively sort the left and right sides of x



Basic idea in sorting an array

- Choose a pivot element from the array
- Start from second element and keep on moving towards right till an element greater than pivot found.
- Start from last element and keep on moving towards left till an element smaller than pivot found.
- Exchange these two elements.
- Use some kind of *Divide and Conquer* strategy



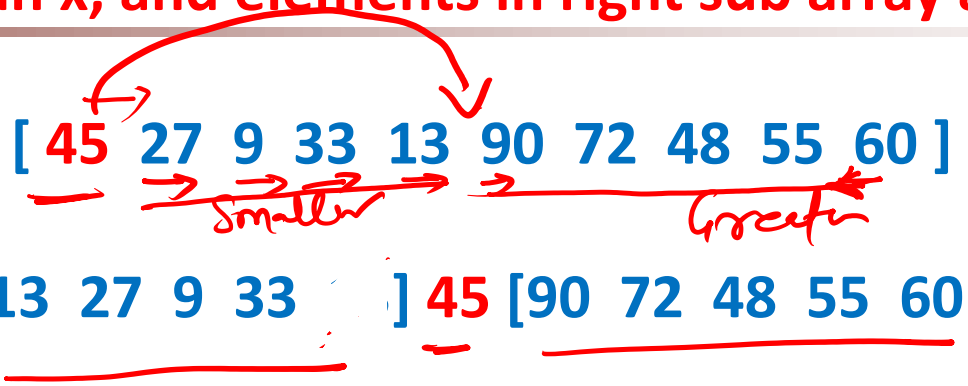
Partitioning an array

Partition Scheme

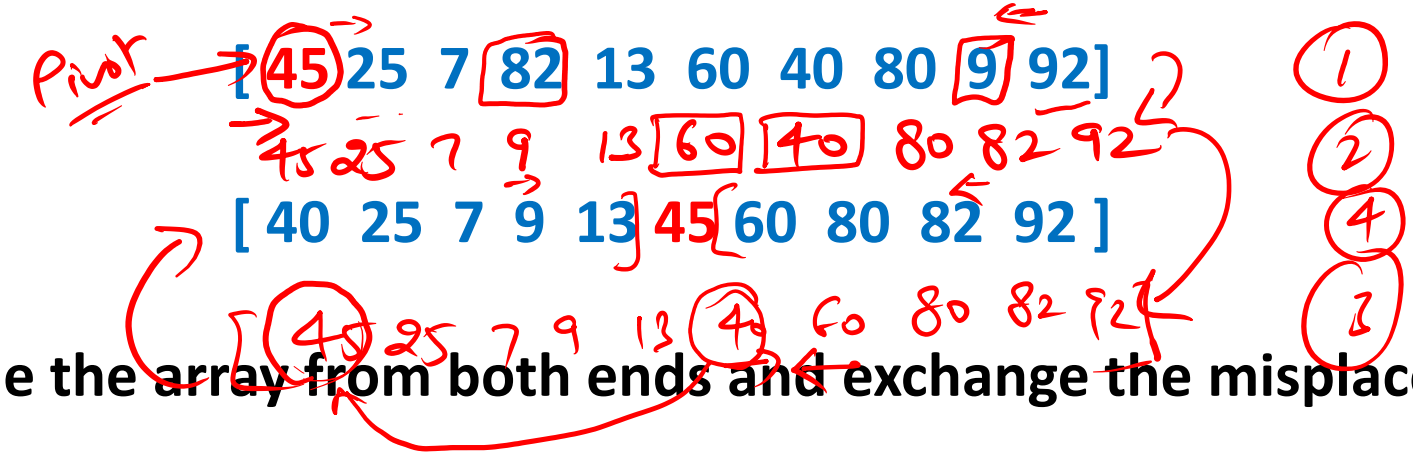
- Hoare partition scheme
- Lomuto partition scheme

Quick - Sort

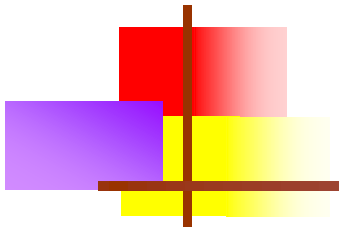
Given x, partition an array in two sub arrays, such that elements in left sub array are smaller than x, and elements in right sub array are greater than x



Order of elements in the sub array : not important



Hint: examine the array from both ends and exchange the misplaced elements



[45 25 7 82 13 60 40 80 9 92]

- examine the array from left and find element not in place

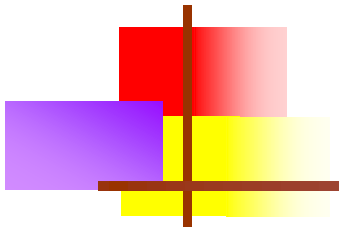
Move to right from A[1] and find element not in place, 82 not in place

- Examine the array from right and find element not in place

Move to left from A[9] and find element not in place, 9 not in place

- Exchange 82 and 9

[45 25 7 9 13 60 40 80 82 92]



[45 25 7 9 13 60 40 80 82 92]

- Continue the process
- Move right from 9 and find element not in place, 60 not in place
- Move left from 82 and find element not in place, 40 not in place
- Exchange 60 and 40

[45 25 7 9 13 40 60 80 82 92]

How long can this process go on?

[45 25 7 82 13 60 40 80 9 92]
l r

[45 25 7 82 13 60 40 80 9 92]
l r

[45 25 7 9 13 60 40 80 82 92]
l r

[45 25 7 9 13 60 40 80 82 92]
l r

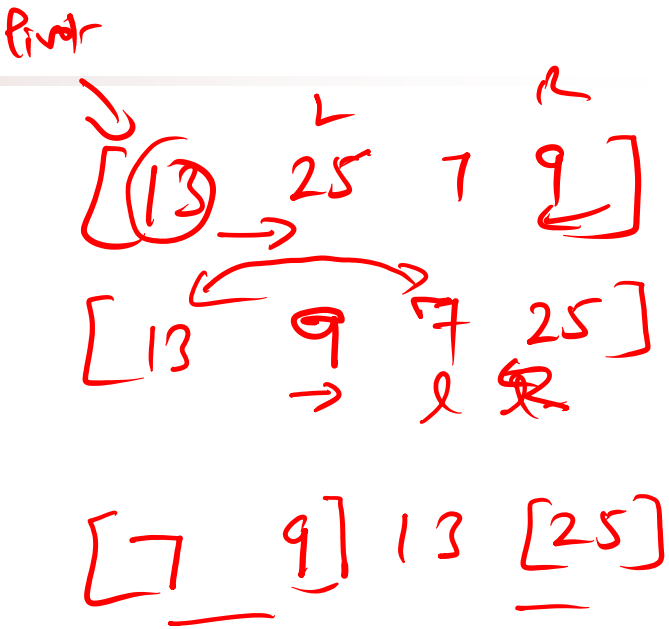
[45 25 7 9 13 40 60 80 82 92]
l r

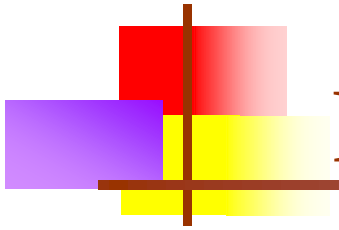
=> r = l-1

Now moving r left will make it smaller than l, so process has to stop here. Finally put 45 in proper place.

[40 25 7 9 13] 45 [60 80 82 92]

[13 25 7 9] 40 45 60 [80 82 92]





Recursive solution

- Partition the array
- Get one element in right place
- Sort the left sub-array
- Sort the right sub-array

Quicksort recursive function

```
void quicksort (int *A, int p, int r)
{
        if (p >= r) return;
        m = partition (A, p, r);
        quicksort (A, p, m – 1 );
        quicksort (A, m +1, r);
}
```

[A(p).. . . . . A(m – 1)] A(m) [A(m+1). . . . . .A(r)]

left pivot right

Algorithm for partitioning array [A[p]…….A[r]]

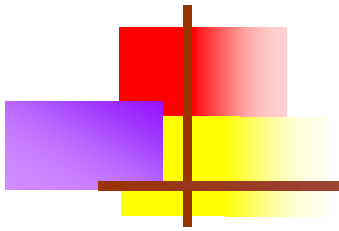
1. **i =** *(first index);* **j =** *(last index),* **pivot =** *(first element);*
2. **while(i < j) do steps 3-8**
3. **while(A[i] <= pivot && i<=r)**
4. **i++;** *(keep moving to right till an element >pivot is found)*
5. **while(A[j] > pivot)**
6. **j – – ;** *(start from last element and keep moving to left till an element <pivot is found)*
7. **if (i < j)**
8. *exchange A[i] with A[j];*
6. **exchange A[j] with A[pivot];** *(This will put pivot in proper place in the array)*
7. **output j as partitioning point.** *(return the new position of the pivot)*

Source Code-Hoare's Partition Scheme

```
public class Quicksort {
    private int[] A;
    private int n;
    public void sort(int[] values) {
          this.A = values;
          n = values.length;
          quicksort(0, n - 1);
    }
    private void quicksort(int low, int high) {
            int i = low+1, j = high, pivot = A[low];
            while (i <= j) {
                while (A[i] < pivot)
                      i++;
                while (A[j] > pivot)
                      j--;
                if (i <= j) {      // exchange if needed
                      exchange(i, j);
                       i++;  j--; // As we are done we can increase i and j
                }
            }
```

Source Code

```
                exchange(pivot, j);
                // Recursion
                quicksort(low, j-1);
                quicksort(j+1, high);
        }
        private void exchange(int i, int j) {
                int temp = A[i];
                A[i] = A[j];
                A[j] = temp;
        }
}
```

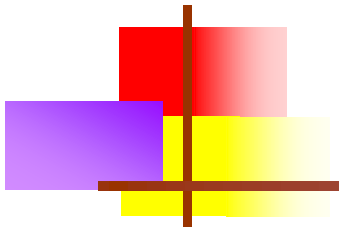


[45 25 7 82 13 60 40 80 9 92]
l r

[45 25 7 82 13 60 40 80 9 92]
l r
[45 25 7 9 13 60 40 80 82 92]
l r
[45 25 7 9 13 60 40 80 82 92]
l r
[45 25 7 9 13 40 60 80 82 92]
l r

While loops are over. Now do r-- . Exchange A[r] with pivot to put 45 in proper place.

[40 25 7 9 13 45 60 80 82 92]



[40 25 7 9 13] 45 [60 80 82 92]

45 is in correct place. Two sub-arrays to be sorted now.

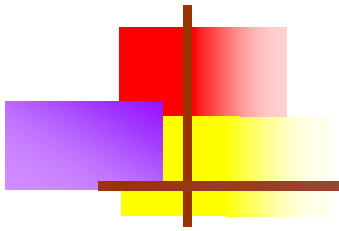
[40 25 7 9 13]45

[13 25 7 9] 40 45

[13 9 7 25] 40 45

[7 9 13] 25 40 45

[7 9] 13 25 40 45



[7 9 13 25 40 45] [60 80 82 92]

[7 9 13 25 40 45] 60 [80 82 92]

[7 9 13 25 40 45] 60 80 [82 92]

[7 9 13 25 40 45 60 80 82 92]

As we are dividing it, we are getting sorted solution

Source Code-Lomuto's Partition Scheme

```
int partition(int arr[], int low, int high)
{
    int pivot = arr[high];    // pivot
    int i = (low - 1);  // Index of smaller element

    for (int j = low; j <= high- 1; j++)
    {
        // If current element is smaller than or equal to pivot
        if (arr[j] <= pivot)
        {
            i++;    // increment index of smaller element
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return (i + 1);
}
```

Animation in the last.

Time Complexity

```
void quicksort (A, p, r){
        if (p >= r) return;
        m = partition (A, p, r);          O(n)
        quicksort (A, p, m – 1 );         T(1)
        quicksort (A, m +1, r);           T(n-1)
}
```

O(n)
T(n/2)
T(n/2)

Time Complexity (Average Case):

$T(n) = 2.\ T(n/2) + O(n)$

$= O(n \log n)$

Time Complexity (Worst Case)

$T(n) = T(n-1) + O(n)$

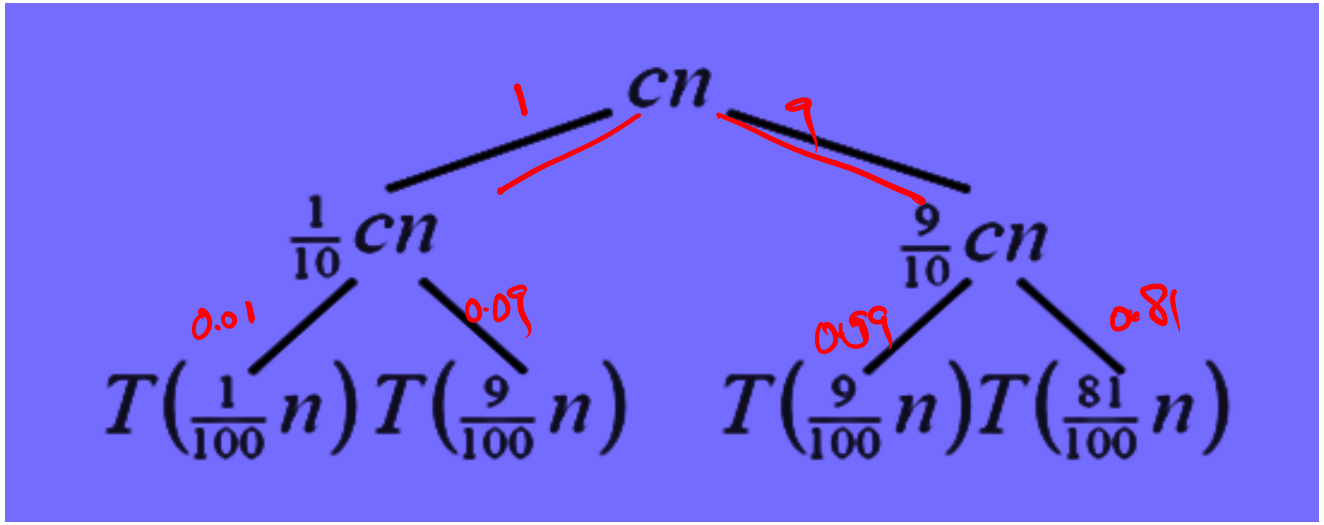
$= O(n^2)$

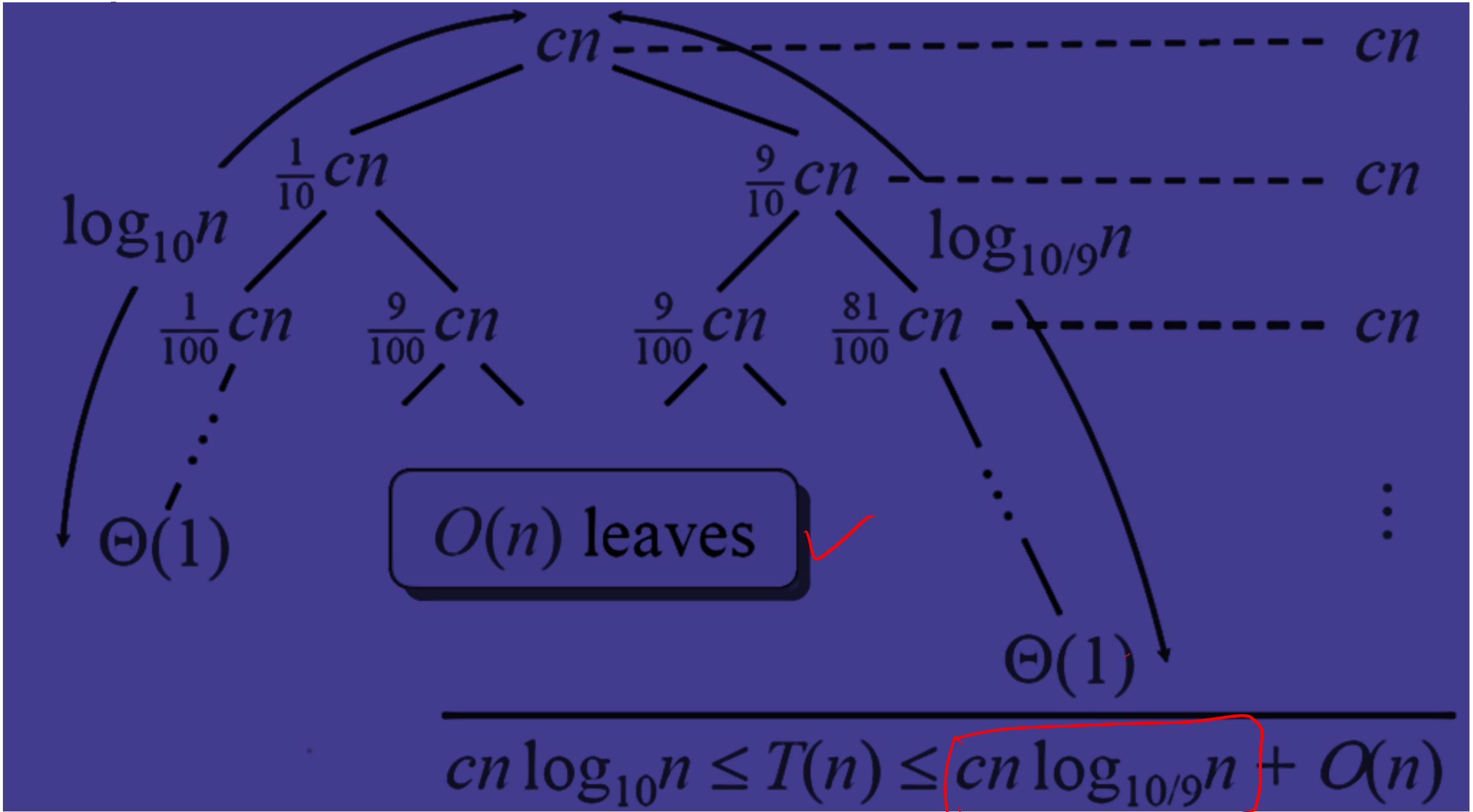
Special Split

What if the split is always $(\frac{1}{10})$ and $(\frac{9}{10})$

$T(n)=T(\frac{1}{10}n)+T(\frac{9}{10}n)+\Theta\, n$

What is the solution to this recurrence?



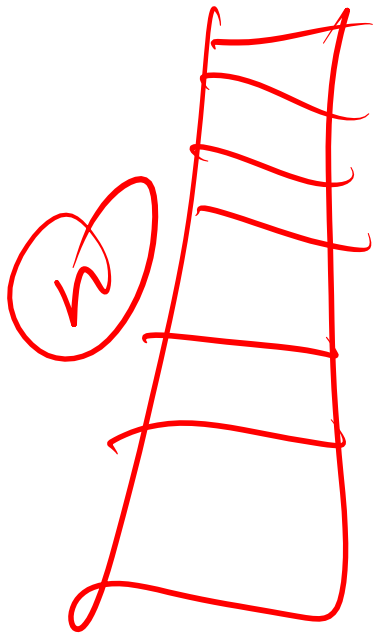

cn
cn
$\frac{1}{10}cn$
$\frac{9}{10}cn$
cn
$\log_{10} n$
$\log_{10/9} n$
$\frac{1}{100}cn$
$\frac{9}{100}cn$
$\frac{9}{100}cn$
$\frac{81}{100}cn$
cn
$\Theta(1)$
$O(n)$ leaves
$\Theta(1)$
$cn \log_{10} n \le T(n) \le cn \log_{10/9} n + O(n)$

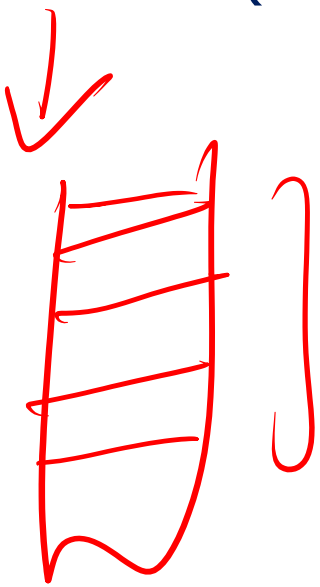
Space Complexity

Worst Case: O(n)

Average Case: O(n)

Best Case: O(logn)



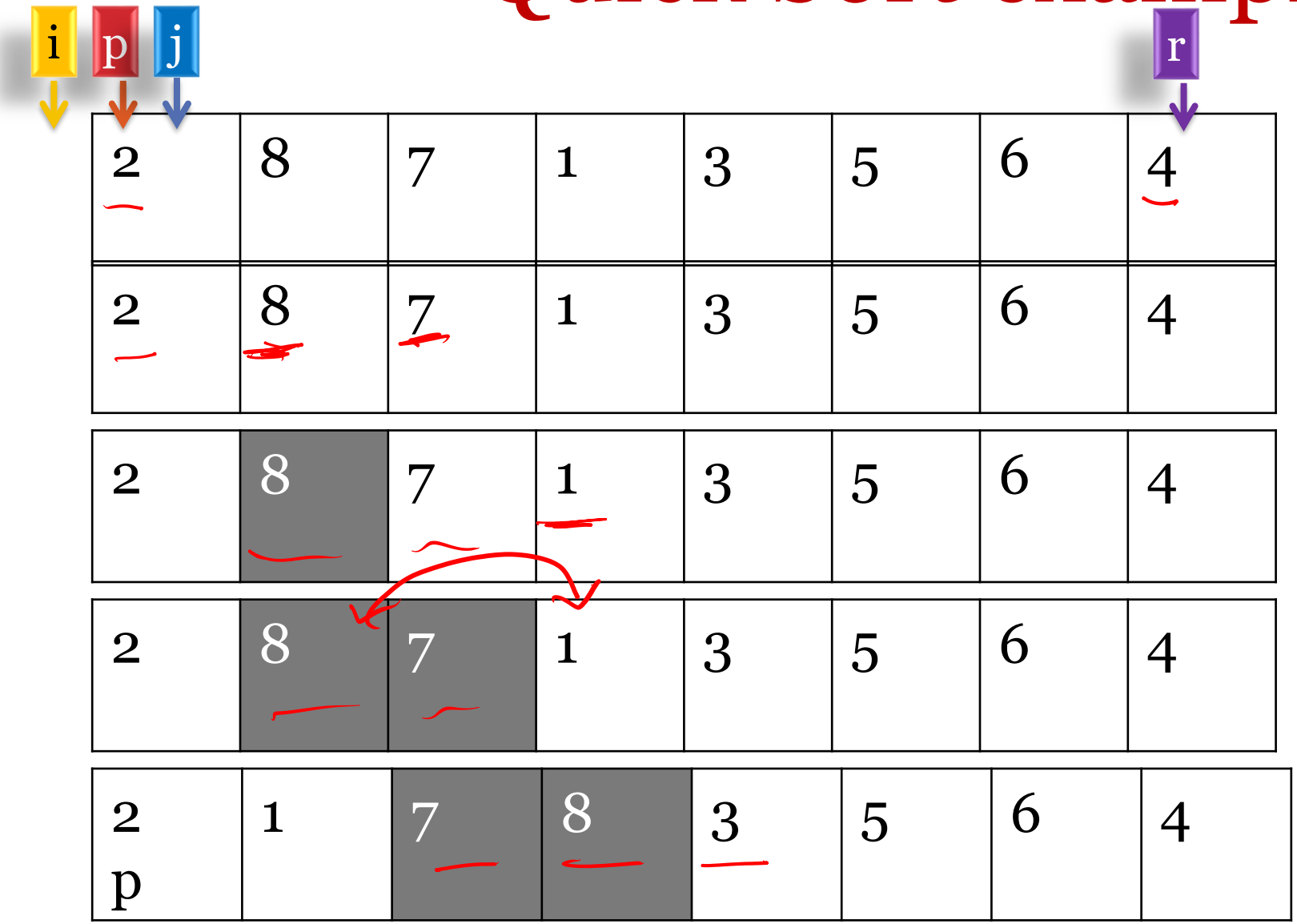


Quick Sort-algorithm

```
p starting index of array A
r  last index of array a
q is the position of pivot after
     partitioning
Quicksort(A, p, r)
if p < r
then q   ← Partition(A, p, r)
Quicksort(A, p, q - 1)
Quicksort(A, q + 1, r)
```

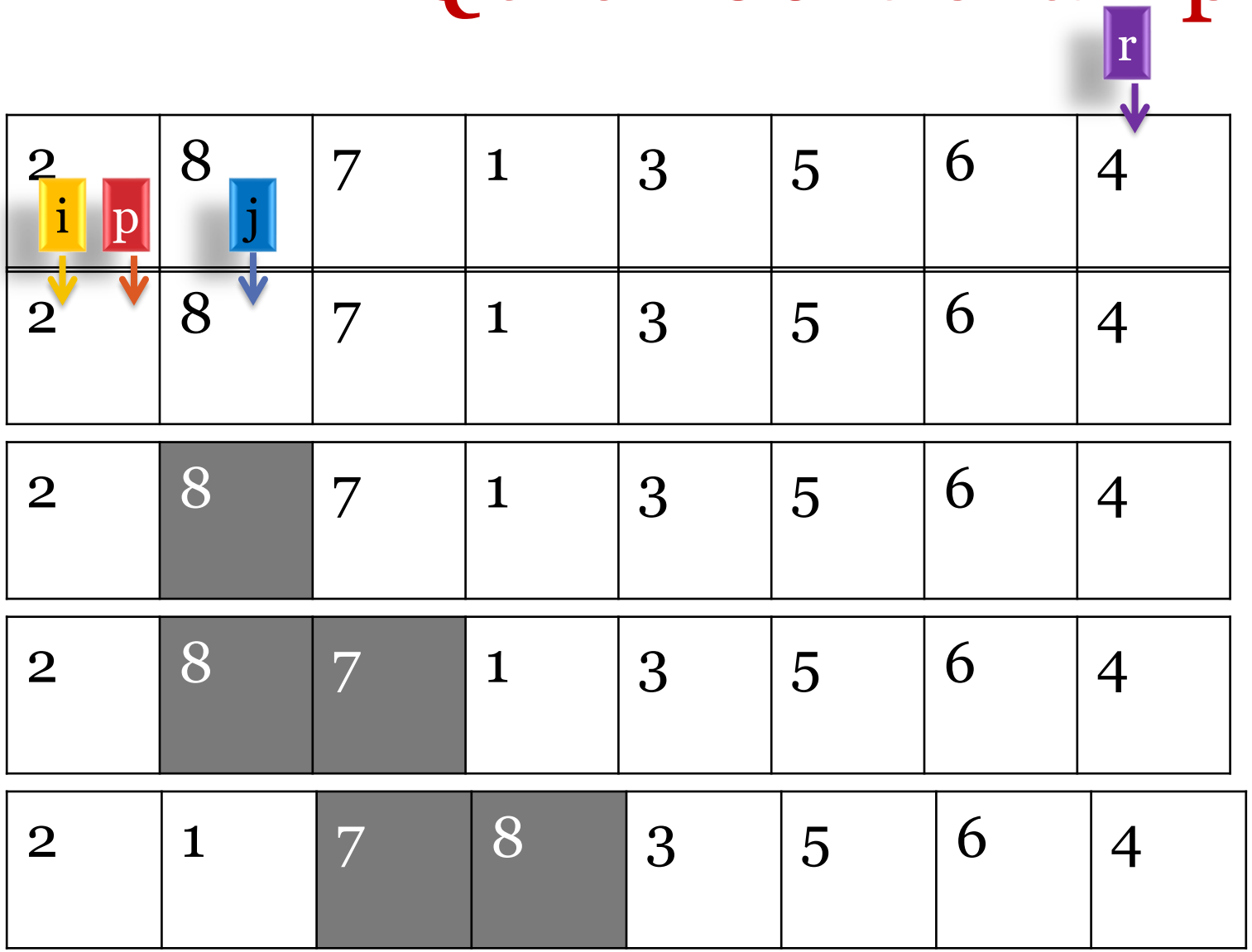
```
Partition(A, p, r)
x ← A[r]
i ← p -  1
for j ← p to r - 1
          do if A[j] ≤ x
                then i ← i + 1
                 swap A[i]  ↔ A[j]
swap A[i + 1] ↔A[r]
return i + 1
```

Quick Sort example (contd)



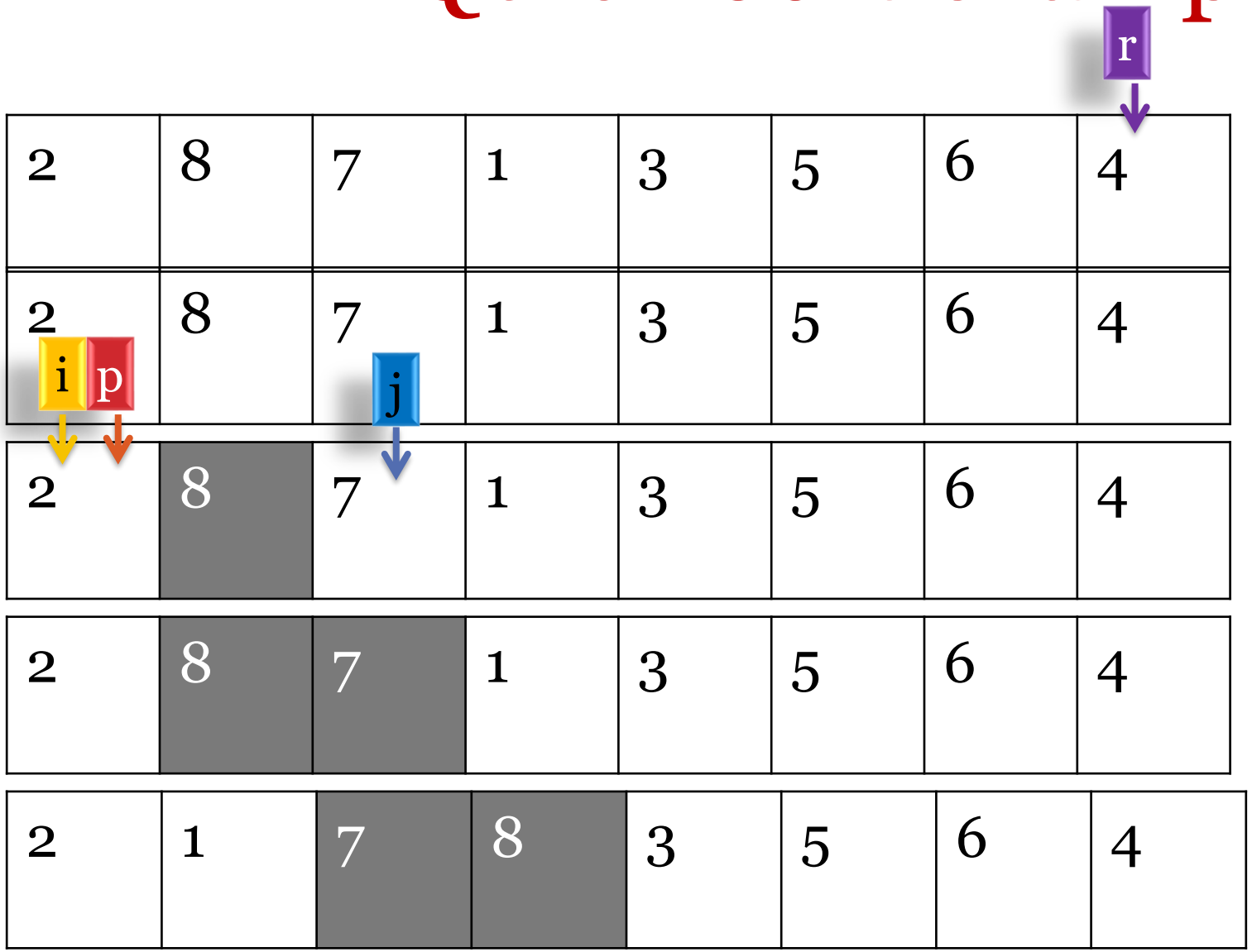

```
Partition(A, p, r)
x ← A[r]
i ← p - 1
for j ← p to r - 1
      do if A[j] ≤ x
      then i ← i + 1
      swap A[i]↔A[j]
swap A[i+1] ↔A[r]
return i + 1
```

Quick Sort example



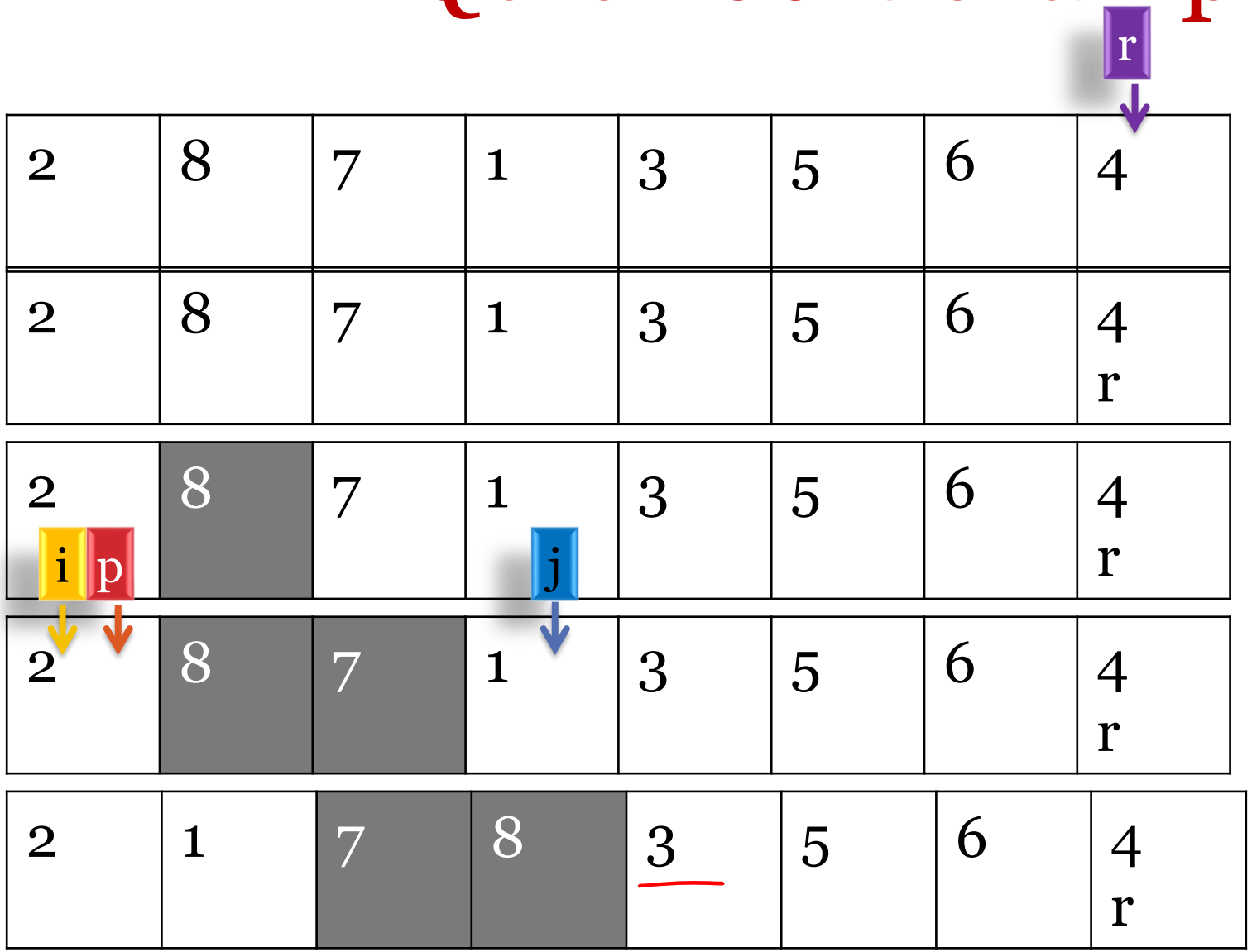

```
Partition(A, p, r)
x ← A[r]
i ← p - 1
for j ← p to r - 1
        do if A[j] ≤ x
        then i ← i + 1
        swap A[i]↔A[j]
swap A[i+1] ↔A[r]
return i + 1
```

Quick Sort example



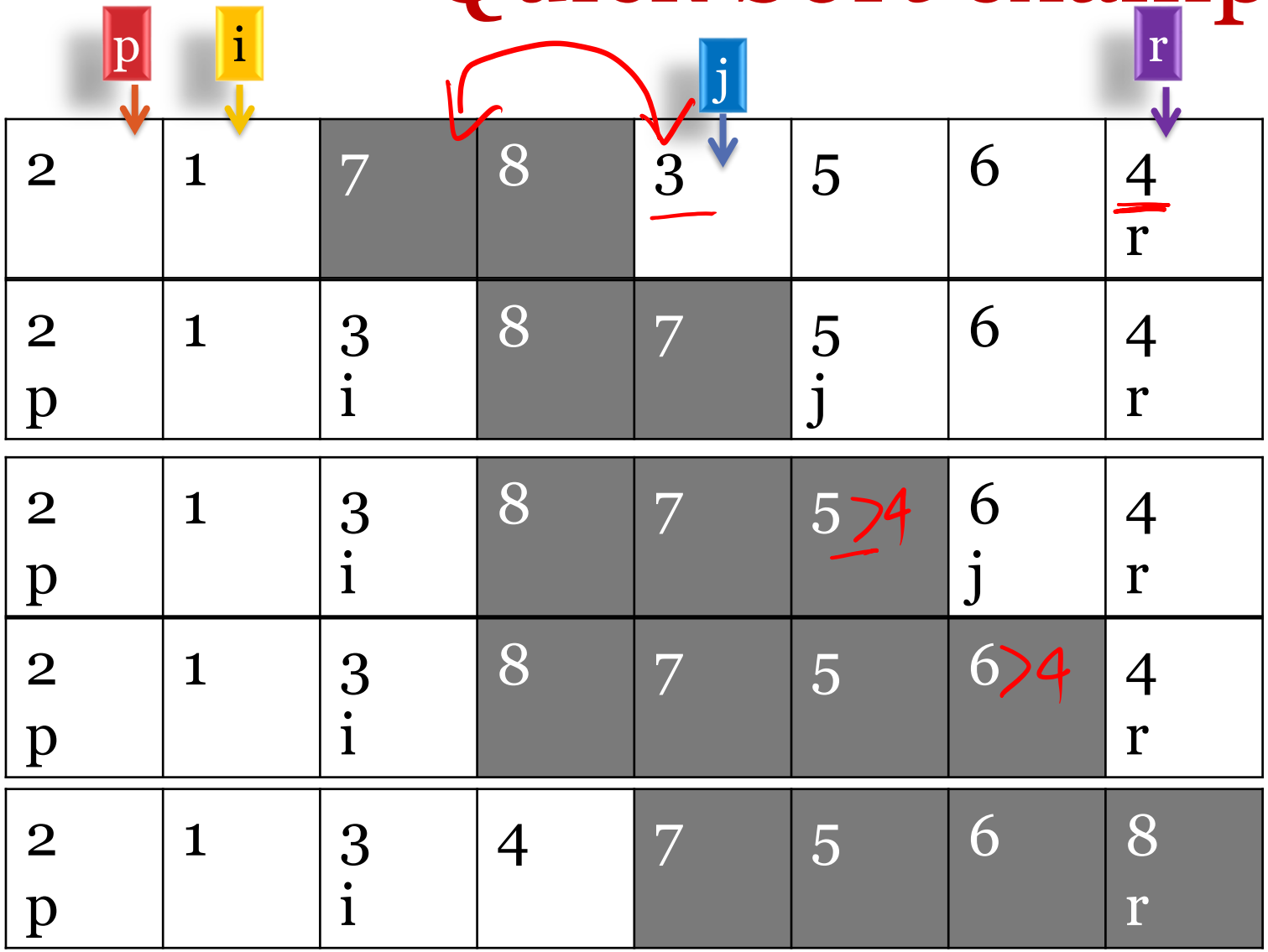

```
Partition(A, p, r)
x ← A[r]
i ← p - 1
for j ← p to r - 1
        do if A[j] ≤ x
        then i ← i + 1
        swap A[i]↔A[j]
swap A[i+1] ↔A[r]
return i + 1
```

Quick Sort example



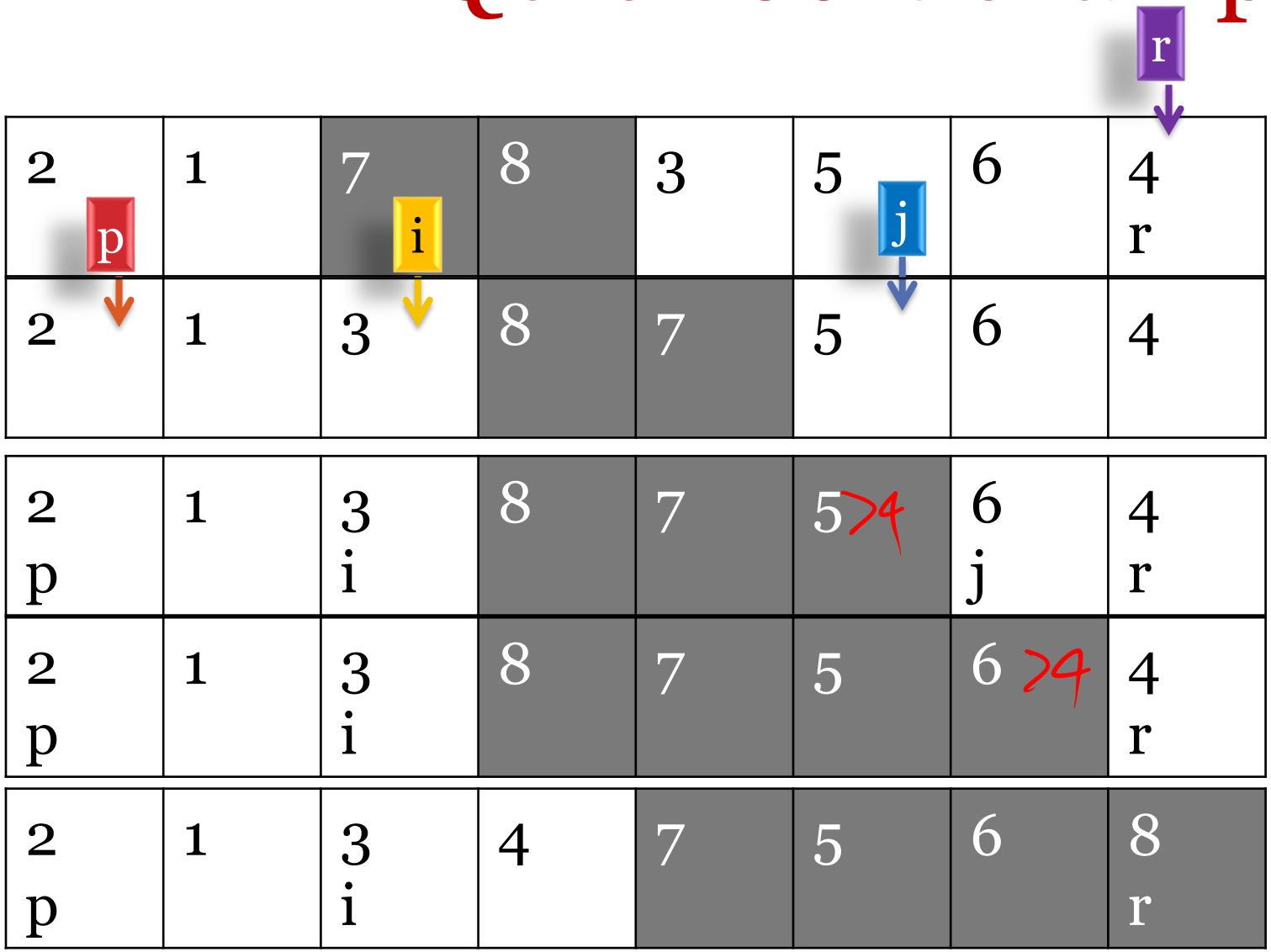

```
Partition(A, p, r)
x ← A[r]
i ← p - 1
for j ← p to r - 1
        do if A[j] ≤ x
        then i ← i + 1
        swap A[i]↔A[j]
swap A[i+1] ↔A[r]
return i + 1
```

Quick Sort example



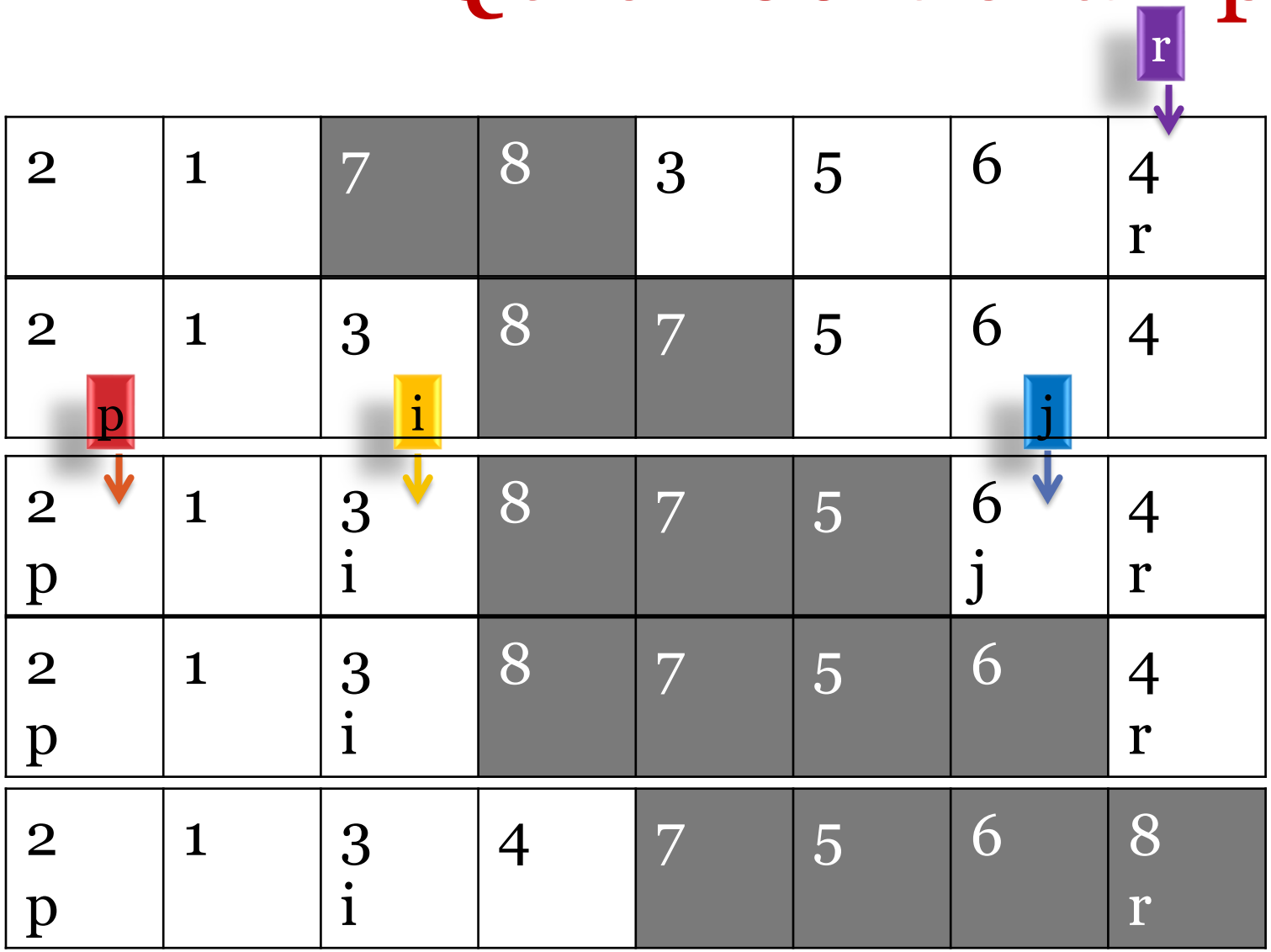

```
Partition(A, p, r)
x ← A[r]
i ← p - 1
for j ← p to r - 1
        do if A[j] ≤ x
        then i ← i + 1
        swap A[i]↔A[j]
swap A[i+1] ↔A[r]
return i + 1
```

Quick Sort example



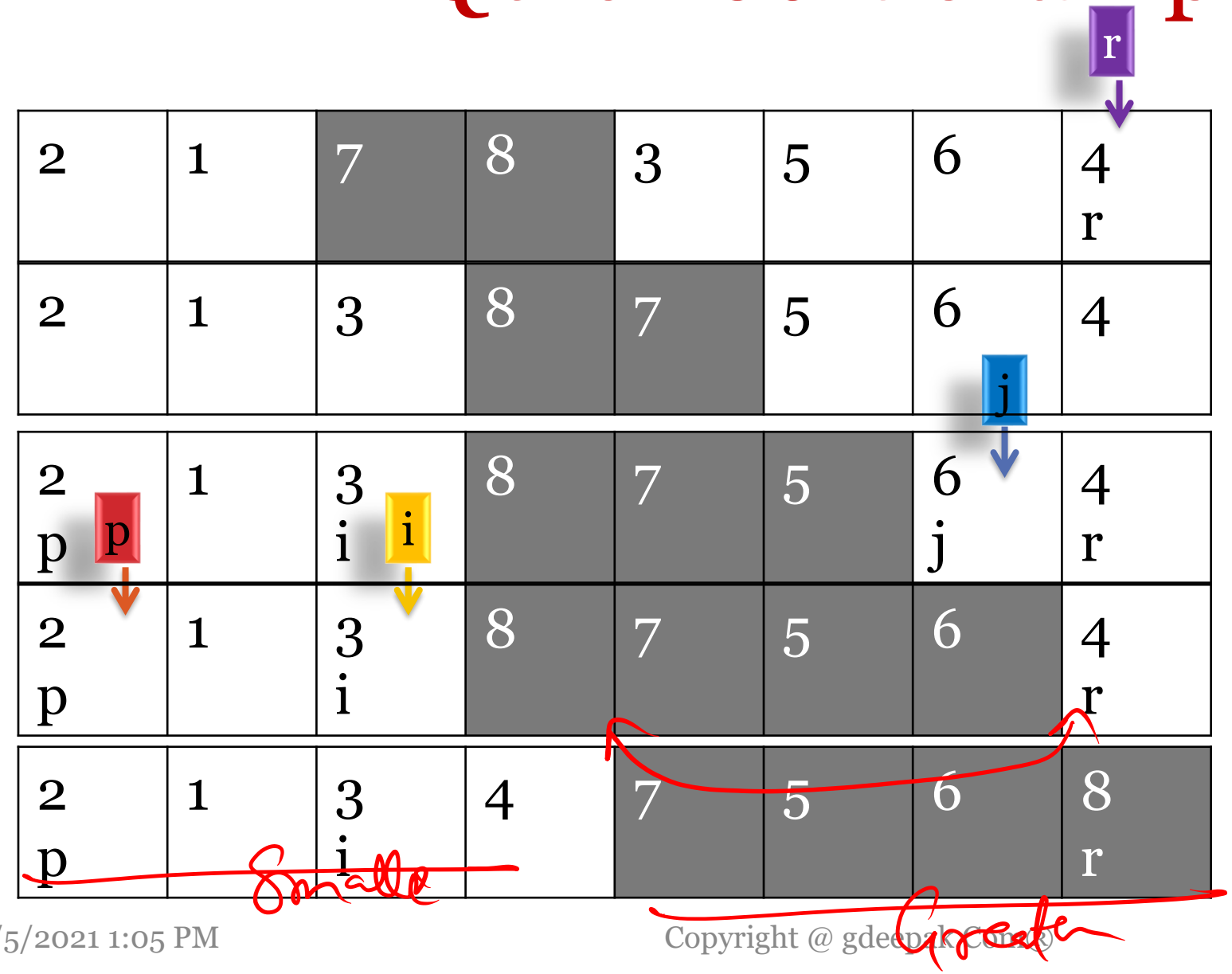

```
Partition(A, p, r)
x ← A[r]
i ← p - 1
for j ← p to r - 1
      do if A[j] ≤ x
      then i ← i + 1
      swap A[i]↔A[j]
swap A[i+1] ↔A[r]
return i + 1
```

Quick Sort example



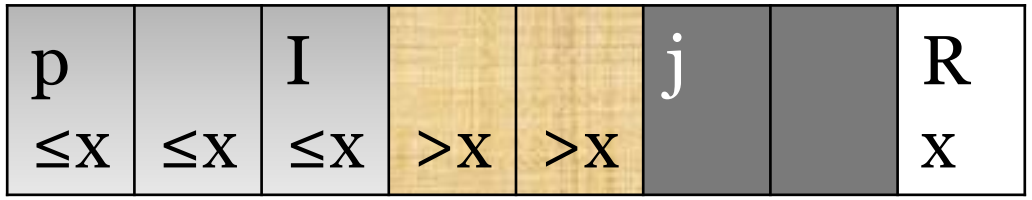

```
Partition(A, p, r)
x ← A[r]
i ← p - 1
for j ← p to r - 1
        do if A[j] ≤ x
        then i ← i + 1
        swap A[i]↔A[j]
swap A[i+1] ↔A[r]
return i + 1
```

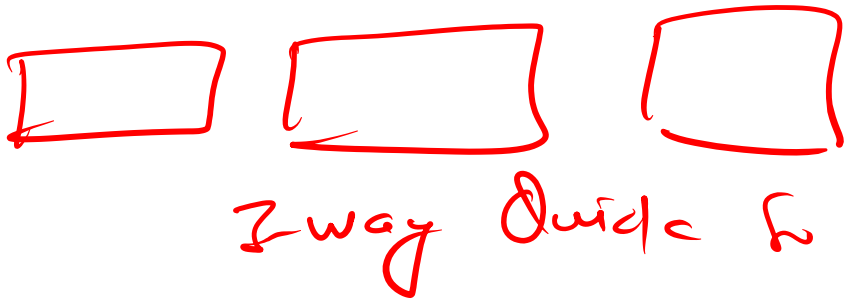
Quick Sort example



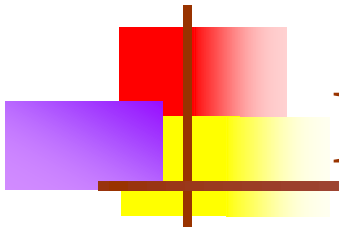

```
Partition(A, p, r)
x ← A[r]
i ← p - 1
for j ← p to r - 1
      do if A[j] ≤ x
      then i ← i + 1
      swap A[i]↔A[j]
swap A[i+1] ↔A[r]
return i + 1
```

Beauty of Quick Sort Invariants





Performance decreases with the presence of large number of duplicates. Need to adopt 3-way Quick Sort. (Advance)



End